

LECTURE 7

JSON & Modern Data Interchange

Parsing, Serialization, and Modern APIs

Today's Agenda

- 1 What is JSON and why it matters
- 2 JSON syntax and data types
- 3 Parsing JSON with Jackson
- 4 Real-world API examples
- 5 JSON vs XML comparison
- 6 Best practices and common pitfalls

PART 1

Introduction to JSON

Understanding modern data interchange

What is JSON?

- JavaScript Object Notation
- Human-readable data interchange format
- Language-independent (works with Java, Python, JavaScript, etc.)
- Lightweight compared to XML
- Native support in all modern programming languages
- Standard format for REST APIs and microservices

JSON Data Types

String

Text in double quotes: "hello"

Number

Integer or decimal: 42, 3.14

Boolean

True or false values

Array

Ordered list: [1, 2, 3]

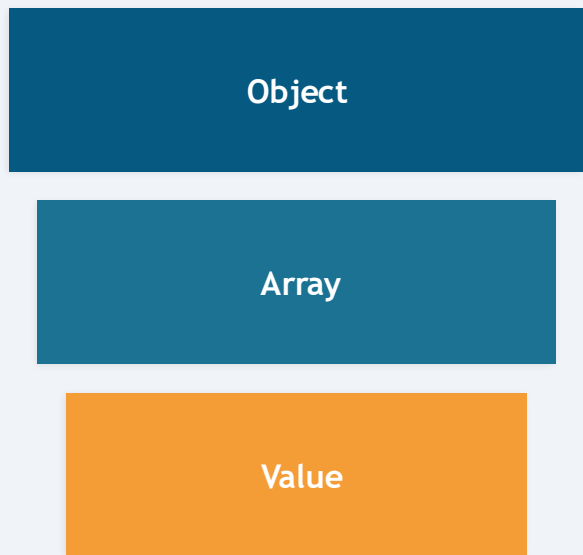
Object

Key-value pairs: {x: 1}

null

Represents absence of value

JSON Structure Hierarchy



- Every JSON document is either an object or array at root
- Objects contain key-value pairs
- Values can be any JSON type (nested structures)
- Arrays contain ordered values
- Primitive values: string, number, boolean, null

Basic JSON Syntax

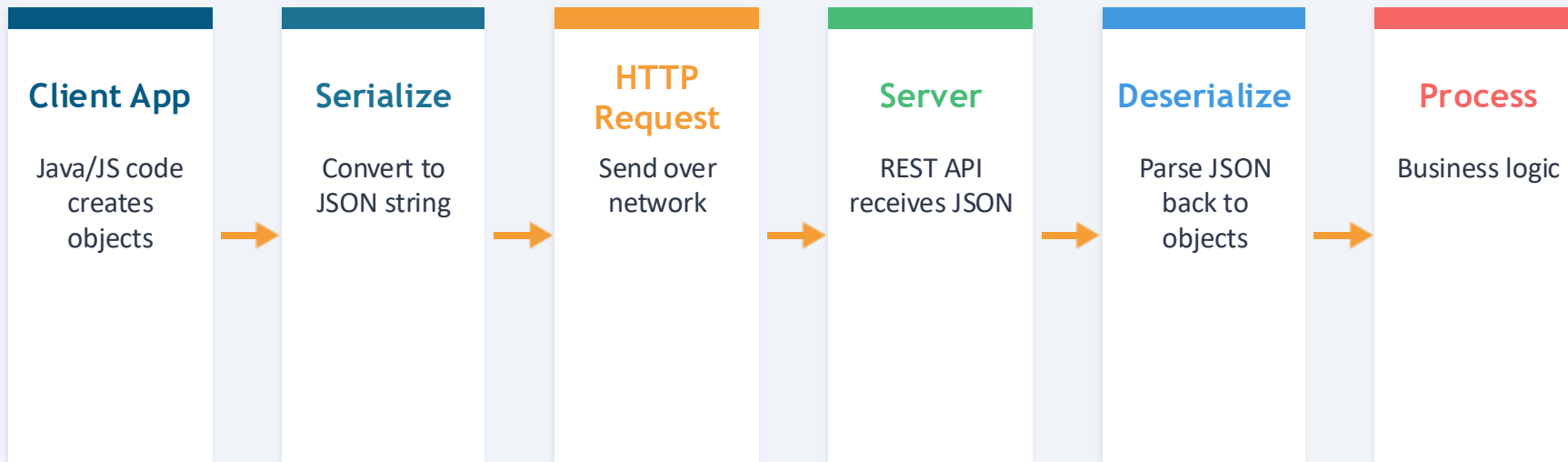
JSON

```
{  
  "name": "John Doe",  
  "age": 28,  
  "isStudent": true,  
  "grades": [95, 87, 92],  
  "address": {  
    "street": "123 Main St",  
    "city": "Boston"  
  },  
  "phone": null  
}
```

Notice: keys must be strings, trailing commas not allowed

JSON Data Exchange Flow

How data moves between client and server



REST API = REpresentational State Transfer + API = Application Programming Interface

A set of rules for how apps communicate over the web — JSON is the most common data format used.

PART 2

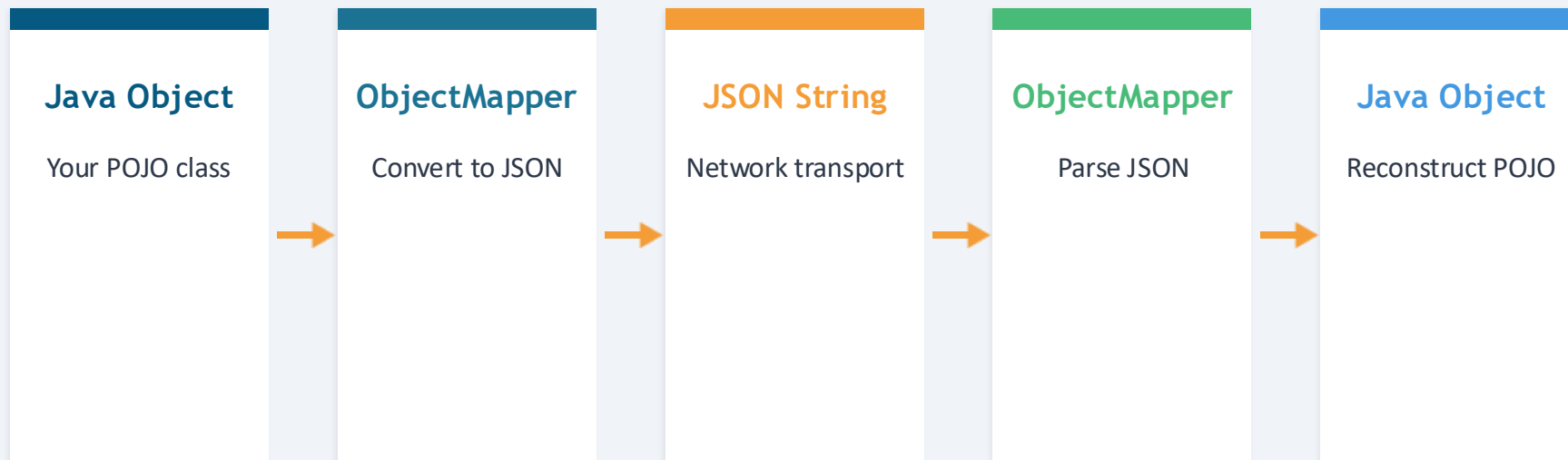
Parsing JSON in Java

Using Jackson library

Jackson Library

- Most popular JSON processing library for Java
- Part of Spring Boot by default
- Fast, efficient streaming parser
- Handles serialization and deserialization
- Supports annotations for customization
- Works with both JSON and YAML

Jackson Library Pipeline



Jackson ObjectMapper Usage

```
ObjectMapper mapper = new ObjectMapper();

// Serialization (Java to JSON)
User user = new User("Alice", 25);
String json = mapper.writeValueAsString(user);
// {"name":"Alice","age":25}

// Deserialization (JSON to Java)
String jsonInput = "{\"name\":\"Bob\",\"age\":30}";
User user2 = mapper.readValue(jsonInput, User.class);
```

ObjectMapper handles conversion between Java objects and JSON

POJO Classes for JSON Mapping

```
public class User {  
    private String name;  
    private int age;  
    private List<String> hobbies;  
  
    // Constructor, getters, setters  
    public User() {}  
  
    public User(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}
```

Field names must match JSON keys exactly (case-sensitive)

Common Jackson Annotations

What is an annotation? A label starting with @ placed above a class, field, or method — it gives instructions to the framework without writing extra code. Think of them as **sticky notes on your code** — Jackson reads them and automatically adjusts how it converts Java ↔ JSON.

- @JsonProperty("customName") - Map JSON key to different field
- @JsonIgnore - Skip field during serialization/deserialization
- @JsonFormat - Control date/number formatting
- @JsonInclude(ALWAYS) - Include null values in output
- @JsonCreator - Custom constructor for deserialization
- @JsonAlias("oldName") - Accept multiple JSON key names

Configuring Jackson

```
ObjectMapper mapper = new ObjectMapper();

// Pretty print output
mapper.writerWithDefaultPrettyPrinter()
    .writeValue(file, user);

// Ignore unknown properties
mapper.configure(
    DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES,
    false
);

// Custom date format
SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd");
mapper.setDateFormat(df);
```

PART 3

JSON in Real-world APIs

Working with REST endpoints

REST API JSON Response

```
GET /api/users/123
```

```
{  
  "id": 123,  
  "name": "Alice Smith",  
  "email": "alice@example.com",  
  "createdAt": "2024-01-15",  
  "posts": [  
    { "id": 1, "title": "First Post", "likes": 42 },  
    { "id": 2, "title": "Second Post", "likes": 28 }  
  ]  
}
```

Parsing API JSON in Java

```
public class UserResponse {  
    private int id;  
    private String name;  
    private String email;  
    private String createdAt;  
    private List<Post> posts;  
  
    // Getters and setters...  
}  
  
// Usage  
String apiResponse = callWeatherAPI();  
UserResponse user = mapper.readValue(  
    apiResponse,  
    UserResponse.class  
);
```

XML vs JSON Comparison

XML (Verbose)

- More verbose syntax
- Larger file sizes
- Steeper learning curve
- Requires XML parser
- Comment support
- Namespaces support



JSON (Clean)

- Compact and readable
- Smaller bandwidth usage
- Easy to learn syntax
- Native JavaScript support
- No comments
- Better API standard

Syntax Comparison: Same Data

XML

- `<person>`
- `<name>John</name>`
- `<age>30</age>`
- `</person>`

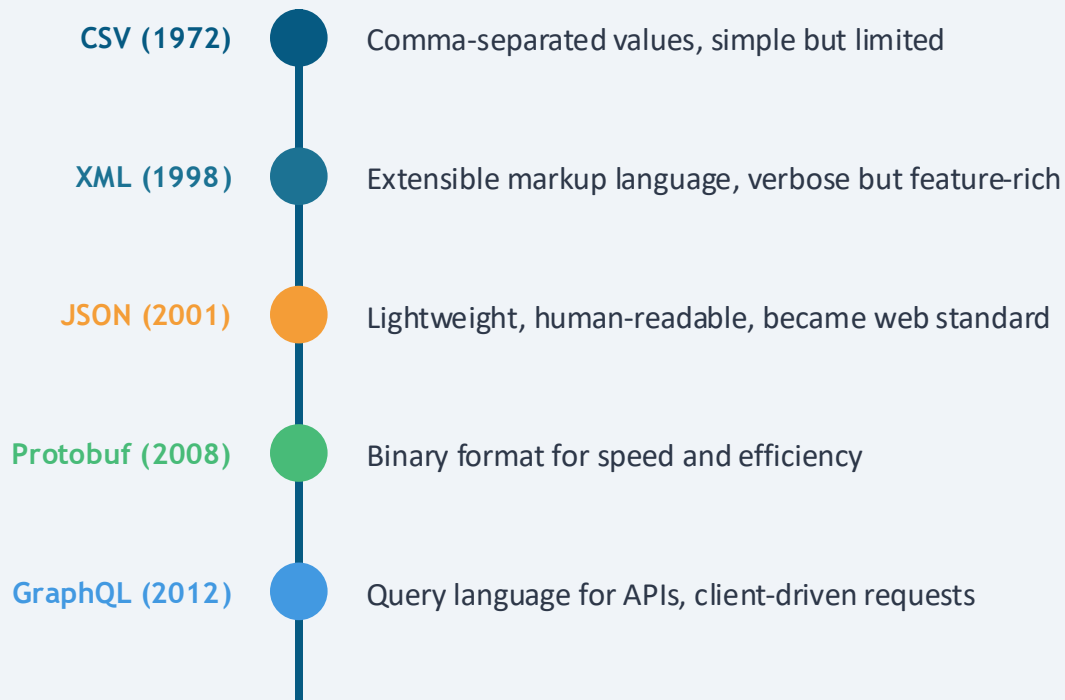
- More markup overhead

JSON

- `{`
- `"name": "John",`
- `"age": 30`
- `}`

- Cleaner and lighter

Evolution of Data Formats



PART 4

Discussion: JSON Parsing Challenge

Hands-on scenario

Parsing Challenge

10 min

SCENARIO

A weather API returns this response:

```
{"temperature": 23.5, "city": "Skopje", "forecast": [{"day": "Mon", "high": 25}, {"day": "Tue", "high": 22}]}
```

Your mobile app needs to display this data.

THINK ABOUT

1

How would you parse this in Java using Jackson?

2

What Java classes would you create to represent this?

3

What if the API adds a new field - does your code break?

Think — Pair — Share

5 min

Case study: Twitter migrated to JSON APIs in 2009

Why would a major platform migrate millions of users from XML APIs to JSON? What are the risks?

1 THINK

Reflect individually for 1 minute

2 PAIR

Discuss with your neighbor for 2 minutes

3 SHARE

Share insights with the class

Hints: Bandwidth savings | Parse speed | Developer experience | Versioning strategy

JSON in Spring Boot — annotations explained simply

How it flows:

CLIENT
sends JSON →

@RequestBody
JSON to Java obj

@ResponseBody
Java obj to JSON

CLIENT
← gets JSON

ObjectMapper
auto-configured

@RestController

Marks a class as a web API endpoint

Every method in it returns JSON automatically — Spring handles all the formatting.

@RequestBody

Incoming JSON → Java object

POST body {"name":"Ana"} automatically becomes a User object in your method.

@ResponseBody

Java object → outgoing JSON response

Return a User object; Spring auto-converts it to {"name":"Ana"} in the HTTP response.

@JsonProperty("name")

Rename a Java field in the JSON output

Java field firstName → "first_name" in JSON. Perfect for matching API naming conventions!

Think of Spring Boot as a waiter: you place an order (JSON in), it brings back food (JSON out).
No manual JSON parsing needed — Spring Boot + Jackson library handles everything automatically.

REST Controller with JSON

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @PostMapping
    public User createUser(@RequestBody User user) {
        // Jackson deserializes JSON to User object
        return userService.save(user);
        // Spring serializes User back to JSON
    }

    @GetMapping("/{id}")
    public User getUser(@PathVariable int id) {
        return userService.findById(id);
    }
}
```

Common JSON Pitfalls in Java

- Field name mismatch (case-sensitive)
- Missing default constructors in POJOs
- Trying to deserialize to final classes
- Circular references causing stack overflow
- Dates without explicit format specifications
- Null handling and optional values

Handling Nulls and Optionals

```
public class Person {  
    private String name;  
  
    @JsonInclude(JsonInclude.Include.NON_NULL)  
    private String phone;  
  
    @JsonSetter(nulls = Nulls.SKIP)  
    private String address;  
  
    // Alternative: use Optional  
    private Optional<String> nickname = Optional.empty();  
}
```

Working with Nested Structures

- Use nested classes for complex objects
- `List<T>` for arrays of objects
- `Map<String, T>` for key-value structures
- Deep nesting can impact performance
- Consider flattening for API design
- Validate nested data integrity

Nested Objects Example

```
public class Company {  
    private String name;  
    private List<Department> departments;  
    private Map<String, Integer> budgets;  
}
```

```
public class Department {  
    private String name;  
    private List<Employee> employees;  
}
```

```
public class Employee {  
    private String name;  
    private double salary;  
}
```

JSON Validation Strategies

- Bean Validation (@Valid, @NotNull, @Email)
- Custom validators for complex rules
- Try-catch for parsing exceptions
- Schema validation before processing
- Input sanitization for security
- Type checking before usage

Validation with Spring Boot

JAVA

```
public class UserRequest {
    @NotBlank(message = "Name required")
    private String name;

    @Min(18) @Max(120)
    private int age;

    @Email
    private String email;
}

@PostMapping
public User create(@Valid @RequestBody UserRequest req) {
    // Spring automatically validates before this method
    return userService.save(req);
}
```


JSON Performance Considerations

- Streaming for large files (JsonParser)
- Field inclusion filters to reduce payload
- Lazy deserialization for optional fields
- Caching ObjectMapper instances
- Compression (gzip) for network transfer
- Consider binary formats (Protobuf) for high volume

Streaming Parser for Large Files

```
JsonFactory factory = new JsonFactory();
JsonParser parser = factory.createParser(
    new File("large_data.json")
);

while (parser.nextToken() != JsonToken.END_OBJECT) {
    if (parser.currentName().equals("user")) {
        parser.nextToken();
        // Process one user at a time, not entire file
        User user = mapper.readValue(parser, User.class);
    }
}
```

JSON Security Best Practices

- Validate all incoming JSON data
- Reject invalid content types
- Use HTTPS for API communications
- Sanitize data before displaying
- Don't expose sensitive data in JSON

PART 5

Advanced JSON Topics

Custom serialization and edge cases

Custom Serializers & Deserializers

```
@JsonSerialize(using = LocalDateSerializer.class)
@JsonDeserialize(using = LocalDateDeserializer.class)
private LocalDate birthDate;

public class LocalDateSerializer
    extends StdSerializer<LocalDate> {

    public void serialize(LocalDate value,
        JsonGenerator jgen, SerializerProvider provider) {
        jgen.writeString(value.toString());
    }
}
```

Handling Polymorphism in JSON

```
@JsonTypeInfo(use = Id.NAME, include = As.PROPERTY,
    property = "type")
@JsonSubTypes({
    @JsonSubTypes.Type(value = Car.class, name = "car"),
    @JsonSubTypes.Type(value = Truck.class, name = "truck")
})
public abstract class Vehicle {
    protected String model;
}
```

API Versioning Strategies

- URL versioning: `/api/v1/users` vs `/api/v2/users`
- Header versioning: `Accept: application/vnd.api.v1+json`
- Parameter versioning: `/users?version=2`
- Deprecated field handling
- Backward compatibility requirements
- Migration planning for major versions

Consuming External JSON APIs

- Use RestTemplate or WebClient
- Handle API rate limits
- Implement retry logic for failures
- Parse error responses
- Cache results when appropriate
- Document API contracts

Key Takeaways

- 1 JSON is the modern standard for data interchange
- 2 Jackson makes Java-JSON conversion simple
- 3 Match POJO structure to JSON schema
- 4 Use annotations for customization
- 5 Validate all incoming data
- 6 Consider performance for large datasets
- 7 Implement proper error handling

Questions?

Next: Lecture 8

CSCI 2450 | Computer Interface Programming | 2026